

Cascading Style Sheets – Level 2



What are *Cascading Style Sheets*?

Cascading Style Sheets (CSS) are a simple (but powerful) method to specify the appearance of HTML pages

- They describe how a page will be displayed on a screen (or printed, or even pronounced by a *non-visual user agent*)
- Their name stems from the fact that several style sheets can be specified for the same web page

Main advantages of style sheets:

- Provide new ways to control page rendering
- Are simple to define
- Can increase the download speed of pages
- Allow more flexible HTML code

CSS have been progressively standardized by the *World Wide Web Consortium* (W3C): CSS1, CSS2, CSS3, ...

Cascading Style Sheets – Levels 1 and 2

Cascading Style Sheets – Level 1 (CSS1)

- W3C official recommendation: December 17th, 1996 (rev. Apr. 11th, 2008)
- Official specification website: <https://www.w3.org/TR/CSS1/>
- Have been the first attempt at separating (as much as possible) the representation style of pages from their content
- Allow *styles* (such as fonts, colors, spaces, etc.) to be associated with HTML pages

Cascading Style Sheets – Level 2 (CSS2)

- W3C official recommendation: May 12th, 1998 (rev. June 7th, 2011)
- Official specification website: <https://www.w3.org/TR/2008/REC-CSS2-20080411/>, <https://www.w3.org/TR/CSS21/> (CSS2.1)
- Are *media-specific*: page presentations can (in principle) be adapted to visual browsers, printers, auditory devices, Braille systems, etc.

How to specify a style sheet for a document?

Three possible methods

1. Direct inclusion in the HTML file

```
<html>
  <head>
    <title>My page</title>
    <style>
      h1 { color: blue }
    </style>
  </head>
  <body>
    .....
    .....
  </body>
</html>
```

2. Link to an external CSS file (best solution)

```
<html>
  <head>
    <title>My page</title>
    <link rel="stylesheet"
          href="mycss.css">
  </head>
  <body>
    .....
    .....
  </body>
</html>
```

3. Integration within a tag

```
<p style="color: red">
```

Until HTML4: attribute `type="text/css"`
necessary for the `<style>` and `<link>`
tags

Basic elements of CSS

Selector

A tag or combination of tags associated with a *declarative block*

Declarative block

A sequence of *declarations* separated by the ‘;’ character and enclosed in curly brackets

Declaration

A *property* followed by the ‘:’ character and by one or more *values* (character strings with associated meanings)

Property

A *feature*

CSS example

```
body {
  color: black;
  font-family: helvetica, sans-serif;
  background: white;
  margin: 2em;
}

h1 {
  margin-top: 2em;
  text-align: center;
}

h2, h3, h4, h5, h6 {
  text-align: left;
}

pre {
  font-family: monospace;
  margin-left: 1em;
  font-size: 90%
}
```

- `body, h1, h2, ..., h6, pre` are *selectors*
- `color, font-family, background, margin, margin-top, text-align, font-family, margin-left` and `font-size` are *properties*
- `black, helvetica, sans-serif, white, 2em, center, left, monospace, 1em` and `90%` are *values*
- `color: black` is a *declaration*
- ```
{
 margin-top: 2em;
 text-align: center;
}
```

 is a *declarative block*

# Document tree of a page

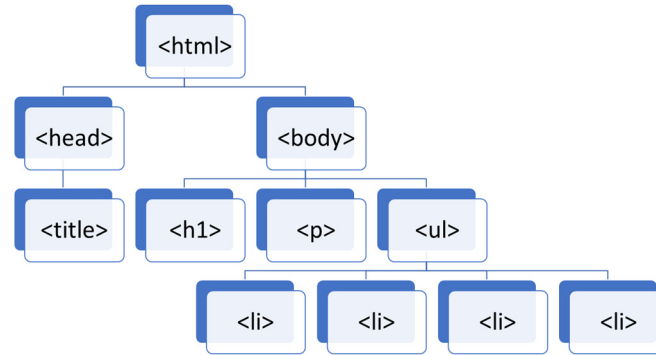
---

Is the hierarchical structure of the page

```
<html>
 <head>
 <title>An HTML page</title>
 </head>
 <body>
 <h1>My page</h1>
 <p>The four seasons:</p>

 Winter
 Spring
 Summer
 Autumn

 </body>
</html>
```



# Selectors...

---

A selector is a tag or a combination of tags

Examples: h1, h2, p, em, pre, body, img, a, h1+h2

If several selectors share the same declarative block, they can be grouped in a list

Example: 

```
h1 { font-family: sans-serif }
h2 { font-family: sans-serif }
```

is equivalent to

```
h1, h2 { font-family: sans-serif }
```

## ... selectors...

---

It is possible to specify the style of a selector according to its hierarchical position in the document tree

```
Example: h1 { color: red }
 em { color: red }
 h1 em { color: blue }
```

means that the color of `<em>` will be red, unless it is inside an `<h1>` tag, in which case it will be blue

It is possible to specify the style of a selector when it is adjacent to another selector

```
Example: h1+h2 { margin-top: -25px }
```

shortens by 25 pixels the vertical space separating an `<h2>` tag from an `<h1>` tag directly preceding it

## ... selectors

---

It is possible to specify the style of a selector for certain attributes and values

Examples: `img[width] { margin: 12px }`

means that if for an `<img>` tag the width is specified (optional `width` attribute), the image will have a 12-pixel margin

`a[href="https://web.unipv.it/"] { color: red }`

means that the links to the website of the University of Pavia will be red

# Classes...

---

Define selector "categories" for which specific styles will be applied

Example: 

```
h1 {
 font-family: Arial, sans-serif;
 color: blue;
}
h1.redtext { color: red }
```

If in the HTML file using this style there is:

```
<h1>Title 1</h1>
```

```
<h1 class="redtext">Title 2</h1>
```

then both "Title 1" and "Title 2" will be written in Arial,  
but the first will be blue and the second will be red

## ... classes

---

It is possible to define generic classes that can be applied to any tag

Example: 

```
.centered {
 text-align: center;
}
```

In any tag, we can write `class="centered"` to center its content (e.g., `<p class="centered">`)

Several classes can be applied at once

Example: `<h1 class="redtext centered">Title</h1>`

"Title" will be red and centered

# Pseudo-classes

---

## Define "states" for HTML tags

Example: `body p:first-child { margin-top: 50px }`  
specifies that the first paragraph of the page will have a 50-pixel upper margin

```
a:link { color: red }
a:visited { color: blue }
```

specify that a link that has not been clicked yet will be red, while an already-clicked link will be blue

hover and  
active can be  
used with any tag

```
a:hover { color: green }
a:active { color: black }
```

specify that, when the mouse cursor is over a link, the link becomes green, and when the cursor is over a link and the mouse button is pressed, the link becomes black

# Pseudo-elements

---

Define the styles for certain parts of the content of HTML tags

**Example:** `p::first-line { text-transform: uppercase }`  
Specifies that the first line of paragraphs will be uppercase

`h1::before { content: "-->" }`  
adds the string "-->" before `<h1>` tags

`h2::first-letter { font-size: 2em }`  
specifies that the size of the first letter of `<h2>` tags will be twice the ordinary character size

# Length units and font size...

---

## *Relative units*

- **em**: relative to the size of the current font (i.e., of the parent element)
- **ex**: relative to the height of the 'x' character of the current font
- **ch**: relative to the width of the '0' character of the current font
- **rem**: relative to the size of the font of the page ("root element")
- **vw**: relative to 1% of the width of the browser window
- **vh**: relative to 1% of the height of the browser window
- **vmin**: relative to 1% of the browser window's smaller dimension
- **vmax**: relative to 1% of the browser window's larger dimension
- **%**: relative to the parent element

## ... length units and font size...

---

### *Absolute* units

- `px`: pixel ( $1\text{px} \cong 1/96^{\text{th}}$  of an inch; for low-resolution screens,  $1\text{px} = 1$  screen pixel; for high-resolution screens and printers,  $1\text{px} =$  more than one screen pixel or point on paper)
- `cm`: centimeters
- `mm`: millimeters
- `in`: inches ( $1\text{in} = 96\text{px} = 2.54\text{ cm}$ )
- `pt`: points ( $1\text{pt} = 1/72^{\text{nd}}$  of an inch)
- `pc`: picas ( $1\text{pc} = 12\text{pt}$ )

## ... length units and font size

---

### Examples:

- `h1 { font-size: 2em }` → the font size is twice the size of the current font (inherited from the parent element)
- `h2 { font-size: 200% }` → like '2em'
- `p { font-size: 20px }` → fixed size of 20 pixels
- `table.halfw { width: 50% }` → tables that use the 'halfw' class occupy, in width, half of their parent element's width
- `table.prop { width: 30vw; height: 30vh }` → tables that use the 'prop' class occupy, in width and in height, 30% of the browser window's width and height, respectively

# Colors

---

## Values available in HTML 4.0 and later

aqua, black, blue, fuchsia, gray, green, lime, maroon, navy, olive, purple, red, silver, teal, white, yellow (and others)

## RGB values

*#hexadecimalRGBvalues, rgb(decimalRGBvalues)*

## Examples:

- `body {color: blue; background-color: yellow }`
- `h1 { color: #ff0000 }`
- `p { color: #f00 }` the same as `#ff0000`
- `em { color: rgb(255,0,0) }`
- `strong { color: rgb(100%, 0%, 0%) }`

# Foreground and background

---

The `color` property defines the color of the foreground (i.e., of text)

Example: `h2 { color: #a2eb53 }`

The `background-color` and `background-image` properties define, respectively, the background color and image

Examples:

- `body { background-color: yellow }`
- `h1 { background-color: rgb(0,0,138) }`
- `body { background-image: url("images/im.jpg") }`

The `background` property can be used for both background color and image

# Properties of background images...

---

## background-repeat

indicates whether the image is repeated both horizontally and vertically (`repeat`, default), horizontally only (`repeat-x`), vertically only (`repeat-y`), or not repeated (`no-repeat`)

## background-attachment

indicates whether the image will scroll with the page (`scroll`, default) or will remain fixed (`fixed`) with respect to the "viewport" (i.e., the browser window)

## background-position

indicates the position of the background image

## ... properties of background images

---

### Examples of values for background-position:

<code>x% y%</code>	means that the point of the image given by <code>x%</code> of its width and <code>y%</code> of its height (starting from the left and from the top) will be in correspondence with the point in the padding area given by <code>x%</code> of its width and <code>y%</code> of its height
<code>val1 val2</code>	for example, <code>2cm 3cm</code> means that the upper left corner of the image will be placed 2 cm to the right and 3 cm below the upper left corner of the padding area

<code>top left and left top</code>	the same as <code>0% 0%</code>
<code>top, top center and center top</code>	the same as <code>50% 0%</code>
<code>right top and top right</code>	the same as <code>100% 0%</code>
<code>left, left center and center left</code>	the same as <code>0% 50%</code>
<code>center and center center</code>	the same as <code>50% 50%</code>
<code>right, right center and center right</code>	the same as <code>100% 50%</code>
<code>bottom left and left bottom</code>	the same as <code>0% 100%</code>
<code>bottom, bottom center and center bottom</code>	the same as <code>50% 100%</code>
<code>bottom right and right bottom</code>	the same as <code>100% 100%</code>

Example:

```
body {
 background-position: 25% 50%;
}
```

# Font properties...

---

## font-family

indicates one or more fonts (specific, such as "Tempus Sans ITC", or generic, such as serif, sans-serif, and monospace)

## font-size

indicates the size of the font and can be a relative value, an absolute value, a percentage, or xx-small, x-small, small, medium (default), large, x-large, xx-large, smaller, and larger

## font-style

indicates the "style" of the font and can have the values normal (default), oblique, and italic

## ... font properties

---

### font-weight

indicates the "boldness level" of the font and can have the values `normal` (default), `bold`, `bolder`, `lighter`, or `100`, `200`, ..., `900` (where `400` is like `normal` and `700` is like `bold`)

### font-variant

indicates the "type" of font and can have the values `normal` (default) and `small-caps`

# Text properties...

---

`text-align`

indicates the text alignment and can have the values `left`, `right`, `center`, and `justify`

`text-decoration`, `text-decoration-line`

indicate the kind of text "decoration" and can have the values `none` (default), `underline`, `overline`, and `line-through`

`text-decoration-color`

indicates the decoration color

`text-decoration-style`

indicates the kind of decoration line and can have the values `solid`, `double`, `dotted`, `dashed`, and `wavy`

## ... text properties...

---

`text-indent`

indicates the indentation of the first text line and can have a length or a percentage as a value

`letter-spacing`

indicates the horizontal space between letters and can have `normal` (default) or a length (additional space) as a value

`word-spacing`

indicates the horizontal space between words and can have `normal` (default) or a length (additional space) as a value

## ... text properties

---

text-transform

transforms text to uppercase or lowercase and can have the values `none` (default), `uppercase`, `lowercase`, and `capitalize`

white-space

indicates how spaces are managed and can have the values `normal` (default), `nowrap`, `pre`, `pre-line`, and `pre-wrap`

# Variables

---

Property values can be assigned to variables

Example (global variables):

```
:root {
 --color1: #1e90ff;
 --color2: #ffffff;
}
body { background-color: var(--color1); }
h2 { border-bottom: 2px solid var(--color2); }
```

Example (local variables):

```
div {
 --mycolor: red;
 color: var(--mycolor);
}
```

Why "CSS"?

# The "cascade"

---

A web page's CSS can have three different origins:

- The *author* of the page, who can specify the styles according to his or her preferences
- The *user agent* (typically, the web browser), which uses a default CSS for those properties that have not been defined by the author of the page
- The *user*, who can define his or her own style sheet

The author's CSS "wins" over the user's and user agent's CSS

unless there are `!important` declarations  
(e.g., `color: red !important;`)

Several styles can be associated with the same page